

In-Context Learning for Foundation Time-series Models with Prior-Data Fitted Networks

Long Vu*, Joseph Natter*, Madeline Frank*, Bobin Deng*, Rui Wu*, Xiaofeng Wu†, Kun Suo*

*College of Computing and Software Engineering, Kennesaw State University, Marietta, Georgia, USA

†City University of Macau, Taipa, Macau

{lvu6, jnatter, mfrank64}@students.kennesaw.edu, xiaofengwu@cityu.edu.mo,

{bdeng2, rwu3, ksuo}@kennesaw.edu

Abstract—Inspired by the success of foundation models, a central question in time-series forecasting research is whether models can adapt to unseen data without relying on dataset-specific training. Existing time-series approaches pursue scale but still lack genuine In-Context Learning (ICL). In this work, we show that Prior-Data Fitted Networks (PFNs), by approximating Bayesian inference from a learned prior constructed through synthetic generative processes, can efficiently achieve this missing capability. Building on this, we propose LinearPFN, a forecasting model that conditions test predictions on real historical data as context, performing task-level inference. Rather than pattern-matching a fixed lookback window, it identifies latent generative structure from observed history, in a single forward pass without parameter updates. This is achieved with only half a million synthetic training samples (orders of magnitude fewer than existing zero-shot foundation models) and a minimalist architecture where a single linear embedding layer paired with patch-based representations proves sufficient to capture diverse temporal patterns. Across eight benchmark datasets, LinearPFN achieves competitive performance compared to many state-of-the-art supervised models, without the need for dataset-specific training, demonstrating a practical and efficient route toward foundation time-series forecasting.

Index Terms—Time-series Forecasting, Bayesian Inference, In-context Learning

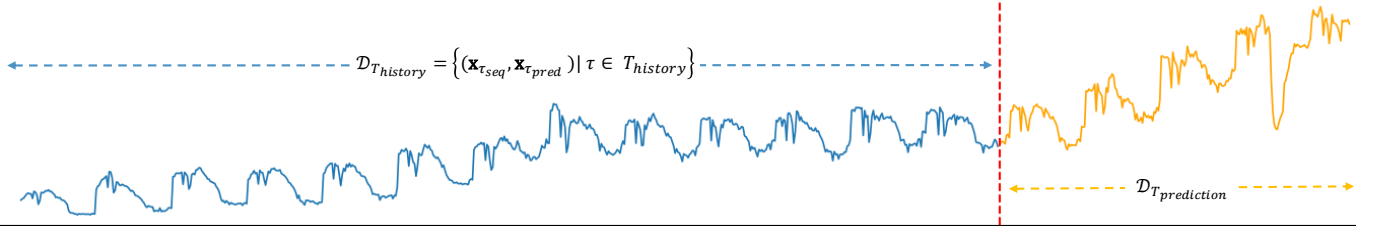
I. INTRODUCTION

Time-series forecasting has been a crucial part of many practical systems: financial risk models, energy demand planning, sensor monitoring, and healthcare analytics. Traditional time-series research has produced a mature set of deep learning models for a wide range of tasks (forecasting, anomaly detection, classification, and imputation) [25]. Recently, research efforts have begun to shift toward foundation models [16], which aim to generalize across unseen data rather than being retrained for each new dataset [3]. They offer advantages over traditional approaches across varied scenarios: real time-series problems span diverse domains, data are often irregular, and task-specific architectures rarely transfer well across different settings. This shift is also influenced by the success of Large Language Models [10], [14], [36], where foundation models demonstrated that scalable, cross-domain models are possible, with *in-context learning* (ICL) emerging as a key mechanism, showing that models can use examples provided at inference time to perform reasoning beyond specific patterns learned in training. ICL has quickly become the core focus of research in recent foundation time-series models [1], [22], [26].

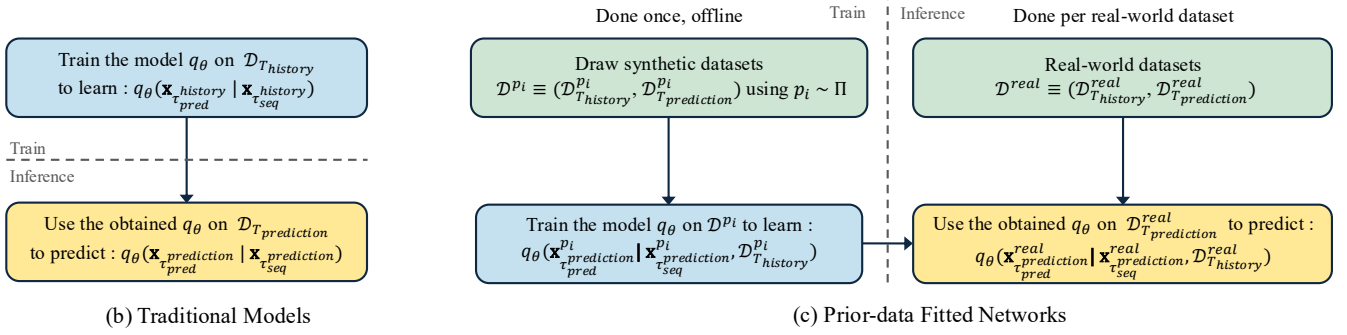
Interest in ICL traces back to observations in GPT-3 [4], later work showed this behavior can be interpreted as implicit Bayesian inference [18], [29]. This observation motivates the PFN framework: rather than seeking generalizable knowledge via model “*scaling*”, PFNs are trained to approximate the Posterior Predictive Distribution (PPD) over tasks by conditioning directly on *context* and *query* sampled from a prior over generative processes [12], [19]. This design makes PFNs an effective route toward explicit ICL. Existing attempts to bring PFNs into time-series forecasting, however, remain incomplete: some never condition on real historical data at inference [2], [6], [21], while others violate the requirement that context and query share a common generative process [23]. These issues prevent these models from using the provided history in a meaningful, principled way. Our work provides a clean formulation of the problem and shows that a properly constructed PFN can perform effective in-context forecasting while being trained on only half a million synthetic samples, substantially smaller than the training requirements reported by many current foundation time-series models [5], [7], [9]. The core differences in principle between PFNs and traditional approaches are highlighted in Figure 1.

With PFNs providing a principled route to ICL, the next question is how to best represent time-series in this process. Recent work suggests that local temporal structure and relative variation are often more informative than absolute position alone [17], [20], [32]. Patch-based methods provide a simple and effective way to exploit this structure [20]. This differs from TabPFN-TS [13], which converts time-series into a tabular problem, treating each time step as an isolated row and using timestamps as features. This discards the data’s natural sequential structure. Guided by these observations, we propose a patch-based representation complemented by a minimal temporal embedding. Motivated by the broader practical lesson that a light temporal embedding layer is often sufficient [22], [32], we use a single linear layer, rather than a more complex embedder, to keep the representation intentionally straightforward. This choice helps ensure that the behavior of the PFN itself remains legible, rather than confounded by embedder complexity. Despite its simplicity, this combination provides stable performance and serves as an effective foundation for ICL-based forecasting.

Overall, our work delivers three valuable contributions:



(a) Example of a target real-world time-series: $\mathcal{D} \equiv (\mathcal{D}_{T_{history}}, \mathcal{D}_{T_{prediction}})$



(b) Traditional Models

(c) Prior-data Fitted Networks

Fig. 1: Difference between traditional time-series forecasting models and PFN models: a) Given a real-world dataset $\mathcal{D}$, the goal is to predict future data $\mathcal{D}_{T_{prediction}}$ using historical dataset $\mathcal{D}_{T_{history}}$. b) Traditional settings use a machine learning model to fit the historical data, then use the obtained model for inference. c) PFN settings fit the model using a broad prior Π that expresses our beliefs about the distribution of every aspects of time-series. At inference, the obtained model uses historical data $\mathcal{D}_{T_{history}}$ as *context* to predict future data using lookback window $\mathbf{x}_{\tau_{seq}}$ as *query* in one single-forward pass, thus achieving in-context learning. * *Note: the superscripts in the figure annotate where the data is drawn from.*

- Foundation time-series models capable of ICL do not require massive pretraining: Existing zero-shot foundation models pursue generalization through scale, requiring pretraining corpora orders of magnitude larger than ours (up to billions of both real and synthetic training samples) [5], [7]. LinearPFN achieves competitive zero-shot performance pretrained on only half a million synthetic samples, demonstrating that Bayesian inference from a learned prior is a more data-efficient path to ICL than scaling model or data size.
- Concrete ICL Formulation in Section III-A: Prior PFN-based time-series methods fail to satisfy PFN’s core requirements. By framing the problem strictly through the PFN lens while maintaining temporal connections within the data, our formulation inherits a mathematically grounded notion of ICL.
- LinearPFN - a lightweight architecture in Section III-D: combining simple patch-based representation with a single linear embedding layer is sufficient for our model to learn various temporal patterns, demonstrating that architectural complexity is not a prerequisite.

Across eight common benchmark datasets, LinearPFN achieves competitive performance compared to established baselines (lower error than fully trained baselines in over half of comparisons, while remaining competitive in the other cases) as shown in Section IV, demonstrating a practical route toward foundation time-series forecasting.

II. RELATED WORK

A. State-of-the-art Time-series Models

Modern time-series forecasting has evolved into a broad field of research [25]. Following the success of LLMs, Transformer-based architectures have played a central role in this development, with early efforts focused on reducing quadratic attention costs or adding inductive biases, including Autoformer [28], Informer [34], FEDformer [35], and Crossformer [33]. Afterward, structural concerns raised by DLinear [32] redirected community focus toward architectures better aligned with temporal structure, driving the adoption of patch-based models (e.g., PatchTST [20], iTransformer [17]), which treat short segments or channels as tokens and emphasize relative temporal patterns. Parallel lines of work explored MLP and CNN backbones (TSMixer [8], TimesNet [27]) and frequency-domain approaches (FreTS [31], FITS [30]). Our work adopts the patch-based paradigm combined with a small linear embedder serving as a feature extractor, as the minimal interface through which the model performs in-context inference over historical data.

B. Foundation Time-series Models

Recent years have seen growing interest in foundation models aiming to mirror the paradigm shift observed in natural language processing: large, generally trained systems that can be applied to new datasets with little or no task-specific retraining. Such models would provide clear practical advantages in

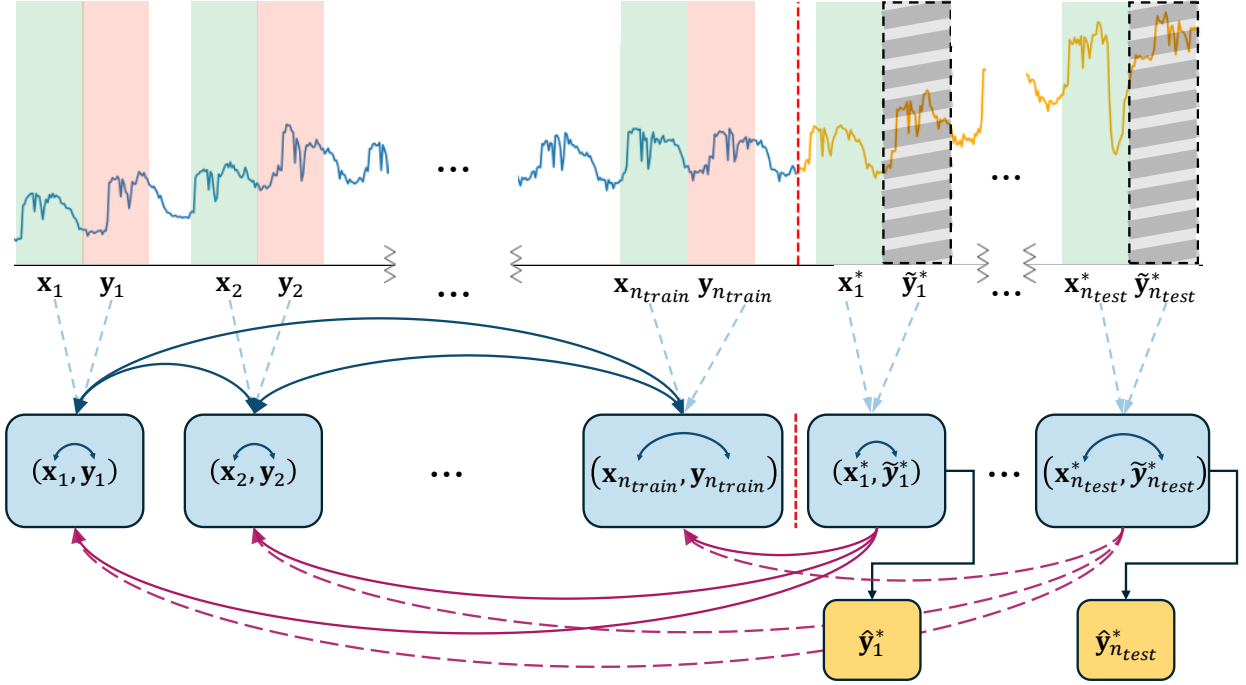


Fig. 2: Core architectural motivation in PFNs: A forecasting task is encoded as a single sequence of context tokens (x_i, y_i) and query tokens (x_j^*, y_j^*) . Query tokens attend to all context tokens to infer the underlying recurring patterns from the observed history, and predictions $\hat{y}_j^*$ are generated in a single forward pass conditioned on this inferred structure.

time-series forecasting [16]. Representative examples include TimeLLM [14], OFA [36], LLMTime [10], and TTM [7]. Despite recent interest, whether current foundation models can solve the core difficulties of time-series forecasting remains a concern [22]. Much of the recent work extends existing architectures primarily through larger training sets and model capacity, offering little to no mechanism for how the model should adapt its predictions once it encounters data it has never seen. As a result, when the temporal structure differs from the pretraining distribution, these models often behave inconsistently, revealing that scale alone cannot substitute for an explicit adaptation strategy [1]. A key element missing from these efforts is a concrete form of in-context learning specific to time-series [26]. This gap motivates the use of Prior-Data Fitted Networks, which offer an explicit Bayesian rationale and forms the basis of our proposed method.

C. Prior-Data Fitted Networks

In-Context Learning, which had long been discussed conceptually as meta-learning [24], was first observed at scale in GPT-3 [4], motivating Bayesian interpretations that view the behavior as implicitly approximating a Posterior Predictive Distribution (PPD) [29]. Müller et al. [18] then proposed Prior-Data Fitted Networks (PFN) to approximate the PPD using a learned model. This idea was later demonstrated in TabPFN [12], showing surprisingly strong empirical results. Theoretical results from [19] clarify the conditions under which PFNs are capable of Bayesian inference, establishing a principled foundation for PFNs. Several works have at-

tempted to extend PFNs to time-series forecasting, beginning with ForecastPFN [6], which introduced a synthetic prior but lacked the structural mechanism required to condition on past observations. Mamba4Cast [2] and TimePFN [21] modify the backbone architecture but inherit the same lack of conditioning on past observations, precluding true task-level inference. LaT-PFN [23] also recognizes this issue and introduces a PFN formulation for time series, but its synthetic prior violates the assumption that samples within a task share a common generative process. The closest implementation to date is TabPFN-TS [13], which transforms each time step into an independent tabular row. This, however, discards essential temporal structure. To the best of our knowledge, this is the first study that adheres to PFN’s theoretical foundation while incorporating inductive biases specifically designed to leverage temporal relationships in time-series forecasting.

III. LINEARPFN: PRIOR-DATA FITTED NETWORKS FOR TIME-SERIES FORECASTING

A. Problem Statement

Consider a univariate time-series forecasting problem: given a sequence $\mathcal{D} = \{y_t\}_{t=1}^T$, the goal is to predict a future sequence $\mathbf{y} \in \mathbb{R}^H$ with horizon of length H , conditioned on past observations. Each supervised instance consists of a lookback window $\mathbf{x} \in \mathbb{R}^L$ and a corresponding target $\mathbf{y}$, yielding a tuple $(\mathbf{x}, \mathbf{y})_\tau$ indexed by time $\tau \in T$.

Most existing methods approach this by learning a dataset-specific regression function $q_\theta(\mathbf{y}|\mathbf{x})$, trained on historical sam-

ples and evaluated on future windows. Under this paradigm, no adaptation occurs at inference time beyond applying the learned mapping encoded in the model’s parameters.

This work proposes a fundamentally different perspective. Rather than treating forecasting as fitting a single global mapping, it can be viewed as a **task-level inference problem**: given a set of past observations, the model should infer the latent generative mechanism responsible for the observed history and condition its predictions on that inferred structure. This viewpoint naturally leads to Bayesian inference and the PPD formulation below.

This work restricts to univariate forecasting to isolate PFN behavior in a controlled setting before extending to higher-dimensional problems. The architecture is not inherently univariate: replacing the scalar patch target with a channel-wise representation is a direct extension, though it requires a corresponding redesign of the synthetic prior to cover cross-channel dependencies.

B. Posterior Predictive Distribution and In-Context Learning

PPDs provide a principled framework for performing in-context learning as Bayesian inference. In classical statistical modeling, data samples $(\mathbf{x}, \mathbf{y})$ are assumed to be i.i.d. draws from an unknown data-generating process p_0 , and prediction is based on estimating $p(\mathbf{y}|\mathbf{x})$. This formulation implicitly assumes a single underlying distribution and does not account for other possible generative mechanisms.

PPDs generalize this view by introducing a prior Π over a family of generative models $\{p_i\}$. From this prior, we sample synthetic tasks $\mathcal{D}^i = (\mathcal{D}_{T_{\text{history}}}^i, \mathcal{D}_{T_{\text{prediction}}}^i)$, each corresponding to one process p_i . The PFN q_θ is trained to approximate the **posterior predictive distribution** π :

$$q_\theta \approx \pi \left(\mathbf{y}_{T_{\text{prediction}}}^i \mid \mathbf{x}_{T_{\text{prediction}}}^i, \mathcal{D}_{T_{\text{history}}}^i \right), \quad (1)$$

amortizing Bayesian inference across Π .

This PPD provides a formal definition of ICL: predictions are conditioned not only on the query input but also on a set of observed context samples, which inform the model about the underlying generative process.

Once trained, the same network can be applied to a real-world dataset $\mathcal{D}^{\text{real}} = (\mathcal{D}_{T_{\text{history}}}^{\text{real}}, \mathcal{D}_{T_{\text{prediction}}}^{\text{real}})$, yielding predictions in a single forward pass:

$$\hat{\mathbf{y}} = q_\theta \left(\mathbf{y}_{T_{\text{prediction}}}^{\text{real}} \mid \mathbf{x}_{T_{\text{prediction}}}^{\text{real}}, \mathcal{D}_{T_{\text{history}}}^{\text{real}} \right). \quad (2)$$

C. Inference Intuition and Architectural Implications

Intuitively, PFNs aim to approximate Bayesian posterior inference across a wide range of possible time-series distributions. Given a new dataset, the PFN infers which patterns from its prior are most likely responsible for generating the observed history, and conditions its prediction accordingly.

Such an architecture requires restructuring an entire forecasting task as a single input (including both historical observations and prediction queries):

$$\mathbf{Z} = ((\mathbf{x}_1, \mathbf{y}_1), \dots, (\mathbf{x}_{n_{\text{train}}}, \mathbf{y}_{n_{\text{train}}}), \mathbf{x}_1^*, \dots, \mathbf{x}_{n_{\text{test}}}^*) \quad (3)$$

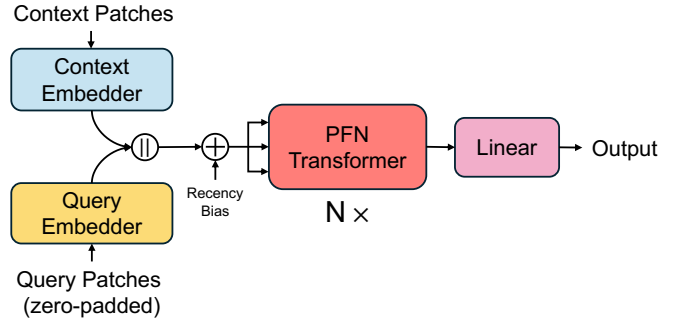


Fig. 3: Architecture design of LinearPFN

Formally, each context token contains both $(\mathbf{x}_i, \mathbf{y}_i)$, while each query token contains only $\mathbf{x}_j^*$ with masked $\tilde{\mathbf{y}}_j^*$. The training part of a real dataset can be viewed as the *context* $\mathcal{D}_{\text{ctx}}$, and each tuple in the test part can be viewed as a *query* $(\mathbf{x}_{\text{qry}}, \mathbf{y}_{\text{qry}})$. The model q_θ then outputs the conditional distributions $q_\theta(\tilde{\mathbf{y}}_j^* | \mathbf{x}_j^*, \mathcal{D}_{\text{ctx}})$, with $j \in \{1, \dots, n_{\text{test}}\}$, $|\mathcal{D}_{\text{ctx}}| = n_{\text{train}}$, computed directly from this input sequence. Figure 2 provides an overview of the architecture. The number of context samples (n_{train}) and queries (n_{test}) can vary between tasks, and the model can accept different values at inference.

D. Architecture Design

Fig. 3 illustrates the overall architecture of our model, following the foundations laid out previously.

1) *Patch Embedding*: Context and query patches are embedded separately, with each embedder taking in the full patch and its time information and projecting it into a shared latent space.

$$e_i = \text{PatchEmbedder}(\mathbf{x}_i, \mathbf{y}_i, \tau_i) \quad (4)$$

The embeddings are then concatenated to form the input sequence

$$\mathbf{Z}^{(0)} = [e_1^{\text{ctx}}, \dots, e_C^{\text{ctx}}, e_1^{\text{qry}}, \dots, e_Q^{\text{qry}}] \in \mathbb{R}^{(C+Q) \times d} \quad (5)$$

2) *PFNTransformer*: The concatenated token sequence is processed by N PFN Transformer blocks:

$$\mathbf{Z}^{(N)} = \text{PFNTransformer}(\mathbf{Z}^{(0)}) \quad (6)$$

The attention mechanism in this paper is constrained by time-series-specific inductive biases: (1) *structural masking* prevents information flow from queries to contexts and between queries; (2) *causal masking* preserves temporal ordering; and (3) an *additive recency bias* encourages attention toward nearby historical tokens while retaining access to the full context. After the PFNTransformer, only query token representations are retained and passed to a linear output head to produce forecasts:

$$\hat{\mathbf{y}}_j = \text{Linear}(\mathbf{Z}_j^{(N)}) \quad (7)$$

TABLE I: Comparison with traditional forecasting models across multiple benchmark datasets. All baseline models are fully trained separately from scratch on each dataset. In contrast, LinearPFN is evaluated in a zero-shot setting: model parameters are fixed, and the training split of each dataset is observed only once at inference without any parameter updates. Best values are bolded and second-best values are underlined.

Model	ECL		ETTh1		ETTh2		ETTh1		ETTh2		Exchange		Traffic		Weather	
	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
Zero-shot																
LinearPFN	0.387	<u>0.310</u>	0.196	0.068	0.275	0.128	0.137	0.034	<u>0.205</u>	<u>0.080</u>	0.267	0.125	0.344	0.249	0.0308	0.015
TimePFN	0.590	0.613	<u>0.186</u>	<u>0.060</u>	<u>0.294</u>	<u>0.143</u>	0.139	0.034	0.251	0.110	0.272	0.126	0.507	0.435	<u>0.0251</u>	0.012
S. Naive	1.017	1.673	0.209	0.072	0.434	0.304	0.141	0.035	0.360	0.231	<u>0.230</u>	<u>0.093</u>	1.557	3.674	0.0257	<u>0.013</u>
Last	0.996	1.635	0.203	0.069	0.423	0.295	0.136	0.033	0.354	0.228	0.221	0.088	1.532	3.628	0.0248	0.012
Mean	0.794	0.965	0.198	0.067	0.359	0.206	0.149	0.038	0.305	0.152	0.302	0.147	1.173	1.862	0.0263	<u>0.013</u>
Supervised																
Informer	0.618	0.639	0.396	0.216	0.412	0.262	0.228	0.082	0.266	0.115	0.785	0.890	0.993	1.436	0.0843	0.106
FEDformer	0.461	0.374	0.216	0.077	0.326	0.171	0.163	0.043	0.225	0.085	0.307	0.157	0.529	0.459	0.0423	0.029
DLinear	0.609	0.611	0.215	0.082	0.345	0.189	0.154	0.041	0.245	0.101	0.310	0.142	0.781	0.887	0.0592	0.057
Crossformer	0.437	0.349	0.328	0.157	0.350	0.190	0.193	0.060	0.223	0.085	0.524	0.412	0.581	0.598	0.0427	0.030
Autoformer	0.569	0.560	0.217	0.077	0.305	0.155	0.180	0.052	0.333	0.155	0.314	0.157	0.489	0.425	0.0777	0.091
PatchTST	0.439	0.363	0.183	0.058	0.308	0.156	0.127	0.029	0.196	0.072	0.235	0.101	0.534	0.499	0.0293	0.016
iTransformer	<u>0.399</u>	0.295	0.195	0.065	0.327	0.174	<u>0.132</u>	<u>0.031</u>	0.216	0.082	0.257	0.119	<u>0.426</u>	<u>0.354</u>	0.0260	<u>0.013</u>

TABLE II: Zero-shot results on univariate time-series forecasting. Errors are averaged over forecasting lengths of {6, 8, 14, 18, 24, 36, 48}. Best values are bolded and second-best values are underlined.

Model	ECL		ETTh1		ETTh2		Exchange		Weather		Traffic	
	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
LinearPFN-36	0.429	0.357	0.146	0.039	0.213	0.081	0.170	0.045	0.2732	0.105	0.400	0.303
TimePFN-36	<u>0.703</u>	<u>0.752</u>	<u>0.130</u>	0.029	<u>0.273</u>	<u>0.126</u>	0.125	<u>0.027</u>	0.1381	0.042	<u>1.032</u>	<u>1.503</u>
ForecastPFN	0.958	1.416	0.237	0.102	0.517	0.434	0.180	0.057	0.1999	0.084	1.742	4.521
Chronos-s	0.792	1.152	0.155	0.061	0.321	0.207	<u>0.113</u>	0.049	<u>0.1136</u>	<u>0.036</u>	1.364	3.103
S. Naive	0.995	1.559	0.151	0.039	0.408	0.279	0.128	0.028	0.1352	0.045	1.771	4.301
Last	0.829	1.211	0.128	<u>0.031</u>	0.336	0.215	0.107	0.022	0.1123	0.035	1.463	3.330
Mean	0.805	0.963	0.154	0.040	0.321	0.168	0.156	0.040	0.1893	0.069	1.256	2.125

IV. EXPERIMENTS

We evaluate LinearPFN on standard long-term forecasting benchmarks and controlled synthetic demonstrations to assess zero-shot performance and in-context reasoning with simple linear embedders.

All experiments use the proposed LinearPFN architecture, with single-layer linear context and query embedders of output dimension $d_{\text{model}} = 512$. The resulting embeddings are processed by a PFN Transformer with 10 blocks ($N = 10$), 8 attention heads ($n_{\text{heads}} = 8$), and feedforward dimension 2048 ($d_{\text{ff}} = 2048$). The model is fully pretrained on a synthetic dataset of half a million time series, each of length 12,000, which is orders of magnitude smaller (well below 1%) than those used by existing zero-shot foundation models [5], [7]. Synthetic series are generated from combinations of long-term linear and exponential trends, multi-cycle seasonal patterns, and signal-dependent noise. Each series uses a fixed

70/30 context/query split analogous to standard train/test splits, with 32 to 1,532 context patches and 1 to 32 query patches; each patch contains a 96-step lookback window and a 96-step prediction horizon. The generator is inspired by LaT-PFN [23], but relaxes its strict multiplicative trend-seasonality interaction by sampling the interaction from a distribution of operators, producing a broader family of generated time series. Reproducible code and model weights are available on <https://github.com/longthangvu/LinearPFN>.

A. Benchmark Evaluation

Evaluation is performed on eight widely used long-term forecasting benchmarks (ECL, ETTh1/2, ETTm1/2, Exchange, Traffic, and Weather) using MAE and MSE [34]. Since the benchmarks are multivariate, we follow prior work and the original TSLib setup by selecting a single target channel for univariate forecasting [25]. Baseline results in Table I are

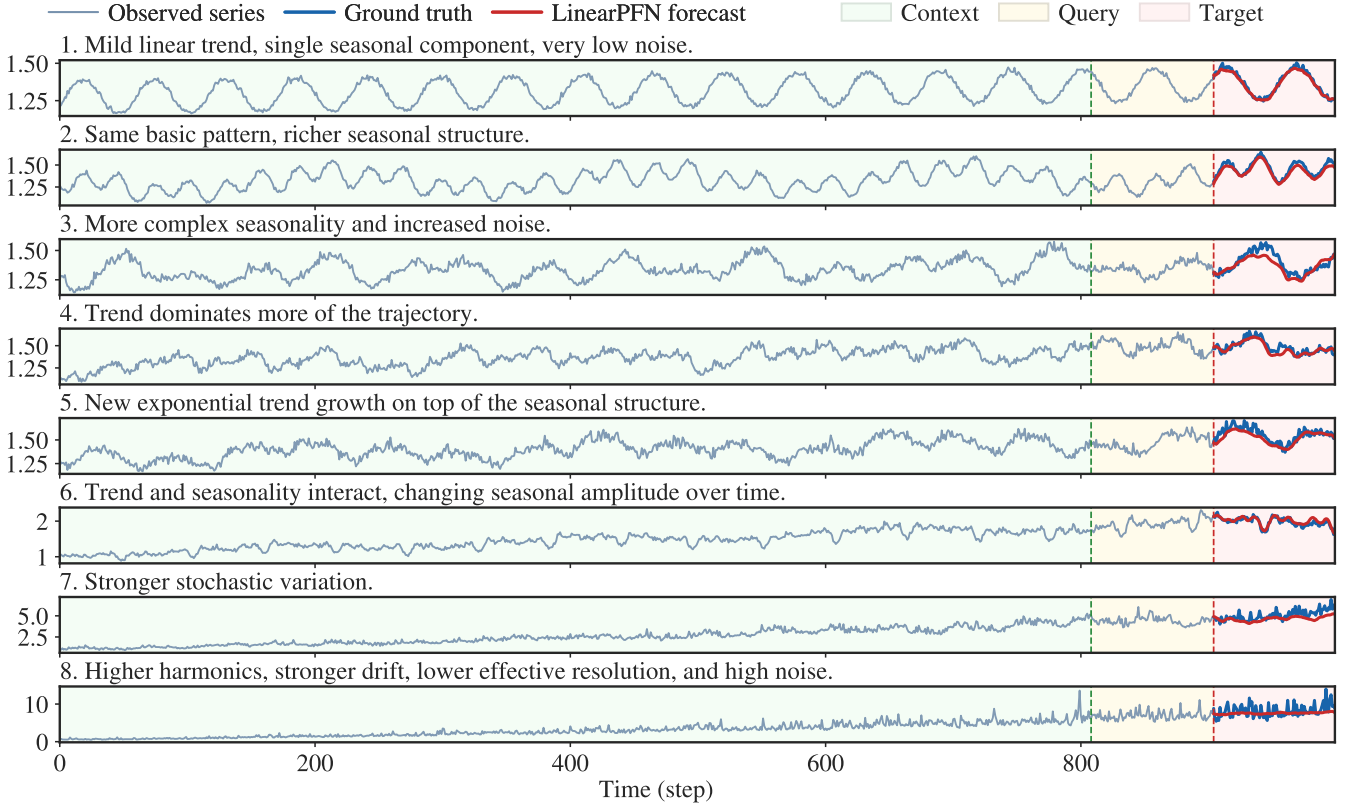


Fig. 4: Qualitative examples from synthetic series where LinearPFN can capture complex patterns not seen in the lookback window. Each example is more complex/noisy than the last: 1) Easy trend + seasonality, 2) More harmonics, 3) Noticeable noise, 4) Larger trend scale, 5) Exponential drift, 6) Multiplicative interaction, 7) Moderate noise, 8) Hardest mixed case

reproduced under this protocol using the TSLib codebase, ensuring a consistent evaluation setting across all models.

A key distinction separates LinearPFN from other zero-shot models in both evaluations: TimePFN and naive baselines treat each test instance independently using only a fixed-length lookback window, while LinearPFN conditions each query on a historical context set constructed from the full training split, enabling in-context inference rather than isolated window-based prediction. Because Transformer self-attention scales quadratically with sequence length, this context cannot be used in full; we apply strided sampling to reduce it to approximately 2,000 patches per evaluation. Although this exceeds the maximum context length seen during pretraining, performance does not degrade, suggesting the model generalizes gracefully to mild out-of-distribution context sizes.

Table I compares supervised baselines trained independently on each dataset under a standard 70/30 split with $sl = pl = 96$ against zero-shot models evaluated without parameter updates, following established conventions in foundation and zero-shot evaluations. LinearPFN achieves lower error than supervised baselines in over half of the comparisons, including strong models such as FEDformer and PatchTST, while relying only on priors learned from synthetic data.

Table II compares LinearPFN with other zero-shot methods

using the TimePFN evaluation pipeline at $sl = pl = 36$ [21]. Because LinearPFN is trained with 96-step patches, we adopt a TimePFN-style truncation strategy for both context and query inputs. LinearPFN achieves the best MSE on ECL, ETTh2, and Traffic, and remains competitive on the remaining datasets. Performance is strongest on datasets with clear global structure where long-range context is informative; on low signal-to-noise datasets such as Weather and Exchange, where naive-last is consistently competitive across methods [1], the context advantage diminishes. Section IV-C examines this behavior directly.

Overall, the results suggest that LinearPFN captures meaningful inductive structure from synthetic priors that transfers well to real-world datasets. The ability to leverage large amounts of contextual information at inference time distinguishes it from traditional zero-shot approaches and contributes significantly to its performance. At the same time, the variability across datasets indicates that there remains room for improvement, particularly in handling highly noisy domains.

B. Qualitative Zero-Shot In-Context Forecasting

Fig. 4 shows zero-shot forecasts on eight controlled synthetic series that isolate different generative factors. The layout separates context (green), query history (yellow), and prediction

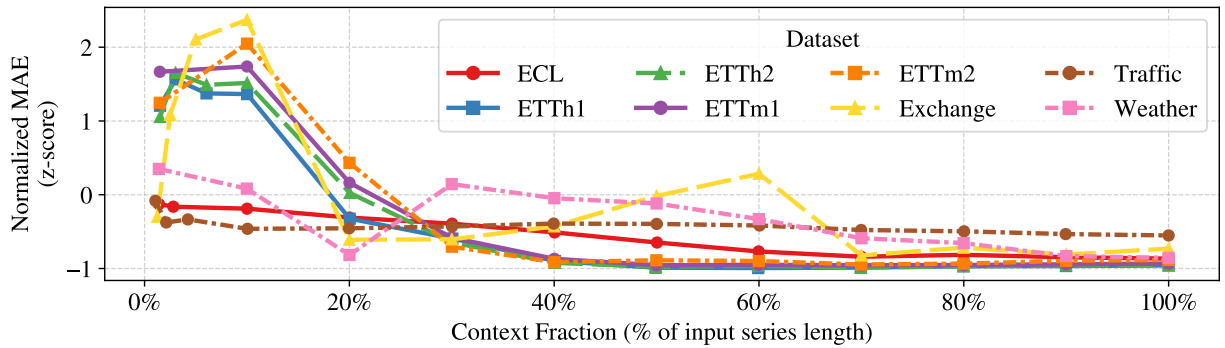


Fig. 5: Effect of context budget on forecasting error across datasets. The x-axis shows the fraction of available context provided at inference time, and the y-axis shows MAE normalized within each dataset using z-score scaling, where lower values indicate better performance. The curves are non-monotonic and dataset-dependent, indicating that larger context budgets do not uniformly improve forecasting accuracy.

target (red), with difficulty increasing from seasonal structure and harmonics to trend, exponential drift, multiplicative interaction, stochastic noise, and a mixed high-noise case.

Across these scenarios, the model extrapolates the dominant structure encoded in the context rather than reacting only to the query window. It recovers phase and amplitude in seasonal cases, preserves non-sinusoidal harmonics, maintains trend and exponential growth behavior, and captures amplitude modulation under multiplicative trend–seasonality interaction. In noisier regimes, predictions track the underlying signal envelope while suppressing high-frequency fluctuations, remaining consistent with the global trajectory inferred from context. These examples illustrate that LinearPFN performs task-level inference conditioned on context patches, composing latent generative components in a single forward pass. This behavior is consistent with posterior predictive inference under a learned prior rather than window-based forecasting.

C. Context Budget Analysis

Because PFN predictions are conditioned on context examples provided at inference time, we can vary the amount of available history and directly measure its effect on forecasting performance. We therefore perform a context budget sweep: for each dataset, we fix the model and evaluation setup while varying the fraction of sampled context provided at inference time, from small prefixes to the maximum sampled context used in the main experiments. Because error scales differ across datasets, Figure 5 reports MAE after within-dataset z-score normalization.

The resulting curves are non-monotonic and dataset-dependent: adding context does not consistently reduce error and can sometimes hurt performance. Unlike standard data scaling, where more data typically gives monotonic gains or diminishing returns [11], [15], PFN context acts as a conditioning signal whose usefulness depends on alignment with the data-generating structure.

At low budgets (roughly 0–20%), most datasets show high error, suggesting that limited history is insufficient to infer reli-

able temporal structure. As context increases, the ETT datasets reach near-optimal performance around 30–40%, while ECL continues to benefit over a wider range, suggesting longer-range dependencies. Traffic becomes relatively flat once a small amount of context is available, whereas Exchange and Weather show more irregular trends, including fluctuations or degradation as additional context is added, consistent with weaker or noisier predictive signals.

Thus, context usefulness depends strongly on temporal structure and signal-to-noise characteristics: beyond a certain point, additional history may be redundant or harmful if it introduces irrelevant or noisy patterns. This also has scalability implications: long-context inference is memory- and compute-limited, and strong performance does not always require the maximum context budget. However, the best cutoff depends on dataset-specific properties that are unknown in a zero-shot setting, so all primary results use the maximum sampled context. Adaptive context selection may therefore improve both efficiency and interpretability by identifying which history segments contribute most to prediction.

High-variability problems (shifting seasonality, special events, structural breaks) are difficult for traditional models because they require dataset-specific architectural modifications. The PFN framework offers a different path: rather than modifying the model, practitioners can manipulate the context directly to encode beliefs about the data-generating process. Targeted context construction and adaptive context selection are therefore natural next directions, with direct relevance to industrial forecasting settings.

V. CONCLUSION

In this work, we introduced LinearPFN, a principled adaptation of Prior-Data Fitted Networks (PFNs) for time-series forecasting. Unlike traditional time-series forecasting models that learn a parametric mapping between historical inputs and future outputs, our approach treats forecasting as in-context inference over task distributions drawn from a synthetic prior. By incorporating temporal structure directly into the PFN

framework, LinearPFN connects Bayesian meta-learning and temporal modeling, enabling generalization across heterogeneous time-series tasks.

We first formulate PFNs for time-series forecasting. This formulation suggests that strong zero-shot performance can be achieved without large-scale pretraining, using a relatively small synthetic corpus, and indicates that architectural simplicity is sufficient for capturing diverse temporal patterns. Building on this foundation, we designed a patch-based embedder that embeds local temporal windows into a unified latent space, allowing the model to capture both short- and long-term dependencies efficiently. The model was meta-trained on synthetically generated datasets constructed with controlled trend, seasonality, and noise components, intended to provide broad inductive coverage. The current parametric prior (combining trend, seasonality, and noise components) covers a broad but bounded family of distributions. Designing more expressive priors is an active problem in PFN research; GP-based constructions as in TimePFN [21] and richer sampling schemes as in TabPFN [12] are candidate directions, though adapting them to the temporal patch structure used here is non-trivial.

Through comprehensive experiments on eight benchmark datasets, LinearPFN showed competitive performance and reasonable generalization while relying on a substantially simpler architecture: a single linear embedding layer rather than the deeper, multi-stage encoders used in comparable transformer-based forecasting models. These results support the viability of PFN-style in-context learning as a scalable and interpretable foundation for time-series forecasting.

ACKNOWLEDGEMENT

We thank the anonymous reviewers for their suggestions and feedback. This research was in part supported by US NSF Grants: SHF-2210744, CSR-2549533, ERI-2501978.

REFERENCES

- [1] Bergmeir, C.: LLMs and foundational models: Not (yet) as good as hoped. *Forecast: The International Journal of Applied Forecasting* **73** (2024)
- [2] Bhethanabhotla, S.K., Swelam, O., Siems, J., Salinas, D., Hutter, F.: Mamba4cast: Efficient zero-shot time series forecasting with state space models. *arXiv preprint arXiv:2410.09385* (2024)
- [3] Bommasani, R.: On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258* (2021)
- [4] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al.: Language models are few-shot learners. *NeurIPS* **33**, 1877–1901 (2020)
- [5] Das, A., Kong, W., Sen, R., Zhou, Y.: A decoder-only foundation model for time-series forecasting. In: *ICML* (2024)
- [6] Dooley, S., Khurana, G.S., Mohapatra, C., Naidu, S.V., White, C.: Forecastpf: Synthetically-trained zero-shot forecasting. *NeurIPS* **36**, 2403–2426 (2023)
- [7] Ekambaram, V., Jati, A., Dayama, P., Mukherjee, S., Nguyen, N., Gifford, W.M., Reddy, C., Kalagnanam, J.: Tiny time mixers (ttms): Fast pre-trained models for enhanced zero/few-shot forecasting of multivariate time series. *NeurIPS* **37**, 74147–74181 (2024)
- [8] Ekambaram, V., Jati, A., Nguyen, N., Sinthong, P., Kalagnanam, J.: Tsmixer: Lightweight mlp-mixer model for multivariate time series forecasting. In: *29th ACM SIGKDD*. pp. 459–469 (2023)
- [9] Garza, A., Challu, C., Mergenthaler-Canseco, M.: Timegpt-1. *arXiv preprint arXiv:2310.03589* (2023)

- [10] Gruver, N., Finzi, M., Qiu, S., Wilson, A.G.: Large language models are zero-shot time series forecasters. *NeurIPS* **36**, 19622–19635 (2023)
- [11] Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L.A., Welbl, J., Clark, A., et al.: Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556* (2022)
- [12] Hollmann, N., Müller, S., Eggensperger, K., Hutter, F.: Tabpf: A transformer that solves small tabular classification problems in a second. *arXiv preprint arXiv:2207.01848* (2022)
- [13] Hoo, S.B., Müller, S., Salinas, D., Hutter, F.: From tables to time: How tabpf-v2 outperforms specialized time series forecasting models. *arXiv preprint arXiv:2501.02945* (2025)
- [14] Jin, M., Wang, S., Ma, L., Chu, Z., Zhang, J.Y., Shi, X., Chen, P.Y., Liang, Y., Li, Y.F., Pan, S., et al.: Time-llm: Time series forecasting by reprogramming large language models. *arXiv:2310.01728* (2023)
- [15] Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., Amodei, D.: Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020)
- [16] Liang, Y., Wen, H., Nie, Y., Jiang, Y., Jin, M., Song, D., Pan, S., Wen, Q.: Foundation models for time series analysis: A tutorial and survey. In: *30th ACM SIGKDD*. pp. 6555–6565 (2024)
- [17] Liu, Y., Hu, T., Zhang, H., Wu, H., Wang, S., Ma, L., Long, M.: itransformer: Inverted transformers are effective for time series forecasting. *arXiv preprint arXiv:2310.06625* (2023)
- [18] Müller, S., Hollmann, N., Arango, S.P., Grabocka, J., Hutter, F.: Transformers can do bayesian inference. *arXiv:2112.10510* (2021)
- [19] Nagler, T.: Statistical foundations of prior-data fitted networks. In: *ICML*. pp. 25660–25676. PMLR (2023)
- [20] Nie, Y., Nguyen, N.H., Sinthong, P., Kalagnanam, J.: A time series is worth 64 words: Long-term forecasting with transformers. *arXiv preprint arXiv:2211.14730* (2022)
- [21] Taga, E.O., Ildiz, M.E., Oymak, S.: Timepf: Effective multivariate time series forecasting with synthetic data. In: *AAAI*. vol. 39, pp. 20761–20769 (2025)
- [22] Tan, M., Merrill, M., Gupta, V., Althoff, T., Hartvigsen, T.: Are language models actually useful for time series forecasting? *NeurIPS* (2024)
- [23] Verdenius, S., Zerito, A., Wang, R.L.: Lat-pfn: A joint embedding predictive architecture for in-context time-series forecasting. *arXiv preprint arXiv:2405.10093* (2024)
- [24] Vilalta, R., Drissi, Y.: A perspective view and survey of meta-learning. *Artificial intelligence review* **18**(2), 77–95 (2002)
- [25] Wang, Y., Wu, H., Dong, J., Liu, Y., Long, M., Wang, J.: Deep time series models: A comprehensive survey and benchmark. *arXiv preprint arXiv:2407.13278* (2024)
- [26] Williams, A.R., Ashok, A., Marcotte, É., Zantedeschi, V., Subramanian, J., Riachi, R., Requeima, J., Lacoste, A., Rish, I., Chapados, N., et al.: Context is key: A benchmark for forecasting with essential textual information. *arXiv preprint arXiv:2410.18959* (2024)
- [27] Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., Long, M.: Timesnet: Temporal 2d-variation modeling for general time series analysis. *arXiv preprint arXiv:2210.02186* (2022)
- [28] Wu, H., Xu, J., Wang, J., Long, M.: Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *NeurIPS* **34**, 22419–22430 (2021)
- [29] Xie, S.M., Raghunathan, A., Liang, P., Ma, T.: An explanation of in-context learning as implicit bayesian inference. *arXiv preprint arXiv:2111.02080* (2021)
- [30] Xu, Z., Zeng, A., Xu, Q.: Fits: Modeling time series with 10k parameters. *arXiv preprint arXiv:2307.03756* (2023)
- [31] Yi, K., Zhang, Q., Fan, W., Wang, S., Wang, P., He, H., An, N., Lian, D., Cao, L., Niu, Z.: Frequency-domain mlps are more effective learners in time series forecasting. *NeurIPS* **36**, 76656–76679 (2023)
- [32] Zeng, A., Chen, M., Zhang, L., Xu, Q.: Are transformers effective for time series forecasting? In: *AAAI*. vol. 37, pp. 11121–11128 (2023)
- [33] Zhang, Y., Yan, J.: Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In: *ICLR* (2023)
- [34] Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., Zhang, W.: Informer: Beyond efficient transformer for long sequence time-series forecasting. In: *AAAI*. vol. 35-12, pp. 11106–11115 (2021)
- [35] Zhou, T., Ma, Z., Wen, Q., Wang, X., Sun, L., Jin, R.: Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In: *ICML*. pp. 27268–27286. PMLR (2022)
- [36] Zhou, T., Niu, P., Sun, L., Jin, R., et al.: One fits all: Power general time series analysis by pretrained lm. *NeurIPS* **36**, 43322–43355 (2023)